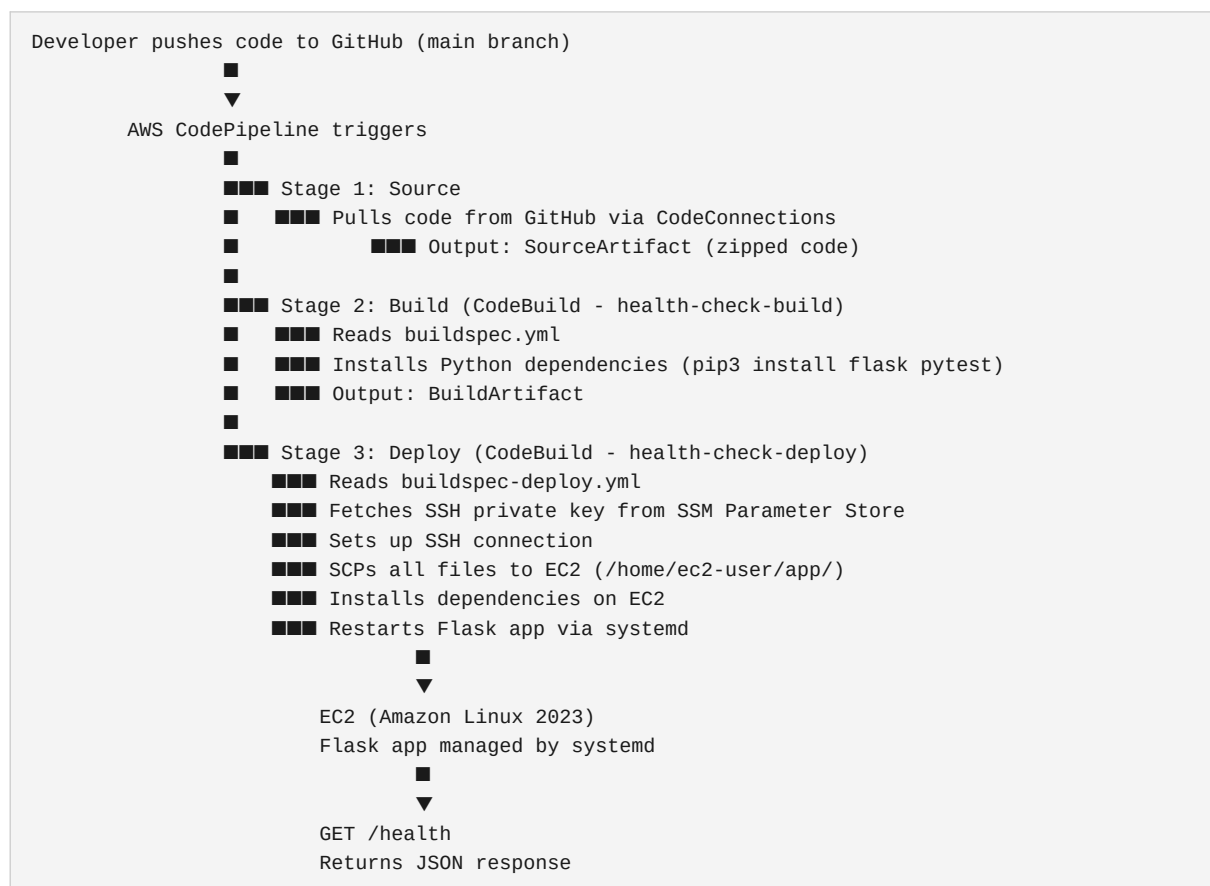


CodePipeline Health Check API — Complete Project Documentation

What We Built

A Python Flask health check API deployed on AWS EC2, with a fully automated CI/CD pipeline using AWS CodePipeline and CodeBuild. The app exposes a `/health` endpoint that returns real-time server status. Every push to the `main` branch triggers the pipeline which builds and deploys the app automatically.

Project Flow (Complete)



Architecture

Component	Service	Purpose
Source control	GitHub	Stores code, triggers pipeline on push
Pipeline orchestrator	AWS CodePipeline	Connects and sequences all stages
Build + test	AWS CodeBuild (health-check-build)	Installs deps, runs build
Deploy	AWS CodeBuild (health-check-deploy)	SSHs into EC2, deploys code
Secret storage	AWS SSM Parameter Store	Stores EC2 SSH private key securely
Compute	AWS EC2 (Amazon Linux 2023)	Runs the Flask app
Process manager	systemd	Keeps Flask app running, auto-restarts on failure
Artifact storage	AWS S3	Stores pipeline artifacts between stages

App — What It Does

A Python Flask API with two endpoints:

GET /health

Returns server health status:

```
{
  "status": "healthy",
  "timestamp": "2026-06-06T11:52:37.970446",
  "uptime": "0h 0m 7s",
  "host": "ip-10-0-8-109.ec2.internal",
  "version": "1.0.1"
}
```

GET /

```
{
  "message": "Health Check API is running",
  "endpoint": "/health"
}
```

File Structure

```
aws-codepipeline-codedeploy/
■■■ app.py                # Flask health check application
■■■ requirements.txt       # Python dependencies
■■■ buildspec.yml         # CodeBuild build stage instructions
■■■ buildspec-deploy.yml  # CodeBuild deploy stage instructions
■■■ appspec.yml           # (kept for reference, not used)
■■■ tests/
■   ■■■ test_app.py       # pytest tests
■■■ scripts/
    ■■■ start.sh          # (kept for reference, not used)
    ■■■ stop.sh           # (kept for reference, not used)
```

All Files — Final Content

app.py

```
from flask import Flask, jsonify
import datetime
import socket
import time

app = Flask(__name__)
START_TIME = time.time()

@app.route('/health', methods=['GET'])
def health_check():
    uptime_seconds = int(time.time() - START_TIME)
    hours = uptime_seconds // 3600
    minutes = (uptime_seconds % 3600) // 60
    seconds = uptime_seconds % 60

    return jsonify({
        "status": "healthy",
        "timestamp": datetime.datetime.utcnow().isoformat(),
        "uptime": f"{hours}h {minutes}m {seconds}s",
        "host": socket.gethostname(),
        "version": "1.0.1"
    }), 200

@app.route('/', methods=['GET'])
def index():
    return jsonify({
        "message": "Health Check API is running",
        "endpoint": "/health"
    }), 200

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

requirements.txt

```
flask==3.0.0
pytest==7.4.0
```

buildspec.yml (final)

```
version: 0.2

phases:
  install:
    commands:
      - echo "Installing dependencies..."
      - pip3 install flask pytest

  build:
    commands:
      - echo "Build successful"

artifacts:
  files:
    - '**/*'
  discard-paths: no
```

buildspec-deploy.yml (final)

```
version: 0.2

phases:
  install:
    commands:
      - echo "Setting up SSH..."
      - mkdir -p ~/.ssh
      - aws ssm get-parameter --name "/health-check/ec2-ssh-key" --with-decryption --query "Parameter.Value" --output text --no-cli-pager --no-cli-pager >> ~/.ssh/ssh_key
      - chmod 600 ~/.ssh/deploy_key
      - ssh-keyscan -H $EC2_HOST >> ~/.ssh/known_hosts

  build:
    commands:
      - echo "Deploying to EC2..."
      - ssh -i ~/.ssh/deploy_key $EC2_USER@$EC2_HOST "mkdir -p /home/ec2-user/app"
      - scp -i ~/.ssh/deploy_key -r * $EC2_USER@$EC2_HOST:/home/ec2-user/app/
      - ssh -i ~/.ssh/deploy_key $EC2_USER@$EC2_HOST "cd /home/ec2-user/app && sudo yum install python3-pip -y && pip3 install flask pytest"

  post_build:
    commands:
      - echo "Deployment complete"
```

Step by Step Setup — Everything We Did

Step 1 — IAM Roles

Created two IAM roles:

EC2-CodeDeploy-Role (attached to EC2 instance)

- Trusted entity: EC2
- Policies: AmazonEC2RoleforAWSCodeDeploy, AmazonS3ReadOnlyAccess, AWSCodeDeployFullAccess
- Purpose: Allows EC2 to communicate with AWS services

CodeBuild-Deploy-Role (used by CodeBuild projects)

- Trusted entity: CodeBuild
- Policies: AmazonS3FullAccess, AmazonEC2FullAccess, CloudWatchLogsFullAccess, AmazonSSMReadOnlyAccess, AWSCodeBuildAdminAccess
- Purpose: Allows CodeBuild to access S3, EC2, SSM, and CloudWatch

Step 2 — S3 Artifact Bucket

Created S3 bucket: **health-check-artifacts-317990**

- Region: us-east-1
- Private (block all public access)
- Purpose: CodePipeline stores artifacts between stages here

Step 3 — EC2 Instance

Launched EC2 with these settings:

- AMI: Amazon Linux 2023 (switched from Ubuntu after CodeDeploy agent issues — see Errors section)
- Instance type: t2.micro
- Key pair: pc-keypair
- IAM instance profile: EC2-CodeDeploy-Role
- Security Group inbound rules:

Type	Port	Source
SSH	22	My IP
HTTP	80	0.0.0.0/0
Custom TCP	5000	0.0.0.0/0

Step 4 — CodeDeploy Agent on EC2 (Amazon Linux)

After switching to Amazon Linux, installed the agent:

```
sudo yum update -y
sudo yum install -y ruby wget
wget https://aws-codedeploy-us-east-1.s3.us-east-1.amazonaws.com/latest/install
chmod +x ./install
sudo ./install auto
sudo systemctl start codedeploy-agent
sudo systemctl enable codedeploy-agent
sudo systemctl status codedeploy-agent
```

Result: active (running)

Note: CodeDeploy agent was installed but ultimately not used — we switched to CodeBuild for deployment instead since CodeDeploy is not in the free tier for on-premises/EC2 in this account setup.

Step 5 — SSM Parameter Store

Stored the EC2 SSH private key securely:

- Path: /health-check/ec2-ssh-key
- Type: SecureString
- Value: full contents of pc-keypair.pem

Why SSM instead of environment variables: When a .pem file is stored as a plaintext environment variable, it loses its newline formatting and the SSH key becomes invalid. SSM Parameter Store preserves the exact formatting.

Step 6 — CodeBuild Projects

health-check-build

- Source: GitHub (Siddhesh-07/aws-codepipeline-codedeploy)
- Buildspec: buildspec.yml
- Role: CodeBuild-Deploy-Role
- Purpose: Install dependencies, run build

health-check-deploy

- Source: GitHub (Siddhesh-07/aws-codepipeline-codedeploy)
- Buildspec: buildspec-deploy.yml
- Role: CodeBuild-Deploy-Role
- Environment variables:
 - EC2_HOST: EC2 public IP
 - EC2_USER: ec2-user
- Purpose: SSH into EC2, copy files, restart app

Step 7 — CodePipeline

Created pipeline: health-check-pipeline

- Pipeline type: V2
- Service role: AWSCodePipelineServiceRole-us-east-1-health-check-pipeline
- Artifact store: health-check-artifacts-317990 (S3)

Stages:

1. Source — GitHub via CodeConnections, branch: main
2. Build — health-check-build CodeBuild project
3. Deploy — health-check-deploy CodeBuild project

Step 8 — systemd Service on EC2

To keep Flask running after SSH disconnects, created a systemd service:

```
sudo tee /etc/systemd/system/healthcheck.service > /dev/null <<EOF
[Unit]
Description=Health Check Flask App
After=network.target

[Service]
User=ec2-user
WorkingDirectory=/home/ec2-user/app
ExecStart=/usr/bin/python3 /home/ec2-user/app/app.py
Restart=always

[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable healthcheck
sudo systemctl start healthcheck
```

Why systemd: Using nohup in an SSH session caused the process to die when the SSH connection closed. systemd manages the process independently of SSH sessions and auto-restarts it if it crashes.

Errors Faced and How We Solved Them

Error 1 — CodeDeploy Agent failing on Ubuntu 22.04

Problem: The CodeDeploy agent install script requires Ruby 2.x or 3.x but Ubuntu 22.04 ships with Ruby 3.3.8 which the script rejected.

Error message:

```
Current running Ruby version for root is 3.3.8, but Ruby version 2.x, 3.x needs to be installed.
```

Things we tried:

- Running with `sudo ruby ./install auto` — same error
- Installing ruby3.0 — package not found on Ubuntu 22.04
- Downloading specific .deb version (1.7.0-52) — 404 not found
- Repackaging .deb with ruby3.0 dependency — ruby3.0 not available

Solution: Switched from Ubuntu 22.04 to Amazon Linux 2023. On Amazon Linux, the install script works cleanly in one command with no Ruby version issues.

Lesson: Always use Amazon Linux for AWS-native tooling like CodeDeploy. Ubuntu has compatibility issues with older AWS tooling.

Error 2 — CodePipeline insufficient permissions for GitHub connection**Error message:**

```
Unable to use Connection: arn:aws:codeconnections. The provided role does not have sufficient permissions.
```

Cause: The auto-created CodePipeline service role didn't have permission to use CodeConnections (GitHub connection).

Solution:

- Added `AWSCodeStarFullAccess` policy to the CodePipeline service role
- Added an inline policy specifically allowing `codeconnections:UseConnection` on the connection ARN

Error 3 — CodePipeline cannot upload to S3**Error message:**

```
Upload to S3 failed: not authorized to perform s3:PutObject
```

Cause: CodePipeline service role didn't have S3 write access.

Solution: Attached `AmazonS3FullAccess` to the CodePipeline service role.

Error 4 — CodeBuild project not found**Error message:**

```
Error calling startBuild: Project cannot be found: arn:aws:codebuild:us-east-1:...:project/health-check-build
```

Cause: The build CodeBuild project (health-check-build) was never created — only the deploy project existed.

Solution: Created health-check-build CodeBuild project separately, then updated the pipeline build stage to reference it.

Error 5 — *pytest not found in CodeBuild (exit status 127)*

Error message:

```
Error while executing command: pytest tests/ -v. Reason: exit status 127
```

Cause: exit status 127 means command not found. pytest was installed but not in PATH.

Things we tried:

- Changed pip to pip3
- Changed pytest to python3 -m pytest
- Changed pip3 to python3 -m pip
- Used absolute pyenv path: /root/.pyenv/versions/3.11.15/bin/pip and /root/.pyenv/versions/3.11.15/bin/pytest

Root cause: CodeBuild's pyenv Python 3.11 installs packages to a different location than where python3 -m pytest looks.

Solution: Simplified buildspec.yml to skip the runtime-version selection entirely and just use pip3 install flask pytest directly. This installs to the system Python which matches the pytest command.

Lesson: CodeBuild's runtime-version selection can cause PATH conflicts. When facing persistent module-not-found issues, removing the runtime-versions block and using system Python is a reliable fix.

Error 6 — *CodePipeline cannot start CodeBuild (deploy stage)*

Error message:

```
not authorized to perform: codebuild:StartBuild on resource: health-check-deploy
```

Cause: CodePipeline service role didn't have permission to trigger CodeBuild projects.

Solution: Attached AWSCodeBuildAdminAccess to the CodePipeline service role.

Error 7 — *SSH key invalid when stored as environment variable*

Error message:

```
Load key "/root/.ssh/deploy_key": error in libcrypto  
Permission denied (publickey)
```

Cause: When a .pem private key is stored as a plaintext CodeBuild environment variable and written to a file, the newlines are lost. An SSH private key without proper newline formatting is invalid.

Solution: Stored the .pem file in AWS SSM Parameter Store as a SecureString. SSM preserves exact formatting including newlines. CodeBuild then fetches it using:

```
aws ssm get-parameter --name "/health-check/ec2-ssh-key" --with-decryption --query "Parameter.Value" --output text >
```

Lesson: Never store SSH private keys or multi-line secrets as plaintext environment variables. Always use SSM Parameter Store or Secrets Manager.

Error 8 — Flask app dying after deploy

Problem: After the deploy stage ran, curl to /health returned connection refused.

Cause: The nohup command used to start Flask was running inside an SSH session. When the SSH session closed after the deploy script finished, the process died.

Things we tried:

- Adding disown after nohup
- Adding sleep 2 before starting

Solution: Created a systemd service (healthcheck.service) on EC2. The deploy script now runs sudo systemctl restart healthcheck instead of nohup. systemd manages the process independently of SSH sessions and restarts it automatically if it crashes.

Lesson: Never use nohup for production processes. Use systemd, supervisor, or pm2 to manage long-running processes.

Error 9 — CodePipeline not auto-triggering on push

Problem: After pushing to GitHub main branch, CodePipeline did not trigger automatically. Required manual Release Change every time.

Investigation:

- GitHub connection status: Available
- DetectChanges: true
- Git trigger configured for push to main
- No webhooks found in GitHub repo settings
- Two different connection ARNs found between Git Triggers and Source action

Attempted fixes:

- Re-saved pipeline to force webhook re-registration
- Deleted and re-added Git trigger
- Verified branch name matches

Status: Unresolved. Pipeline requires manual Release Change to trigger. Likely a GitHub App webhook registration issue specific to this CodeConnections setup.

Workaround: Use Release Change manually, or switch to GitHub Actions for the source trigger (which handles webhooks more reliably).

Error 10 — *git push rejected (non-fast-forward)*

Error message:

```
Updates were rejected because a pushed branch tip is behind its remote counterpart.
```

Cause: Local branch was named master but remote was main. After GitHub made changes to the repo (via console uploads), local was behind.

Solution:

```
git pull origin main --rebase
git branch -M main
git push origin main
```

Lesson: Always check your local branch name matches remote with git status. Use git branch -M main to rename master to main.

IAM Permissions Summary

All permissions added to CodePipeline service role throughout troubleshooting:

- AWSCodeStarFullAccess
- AmazonS3FullAccess
- AWSCodeBuildAdminAccess
- Inline policy: codeconnections:UseConnection

All permissions on CodeBuild-Deploy-Role:

- AmazonS3FullAccess
- AmazonEC2FullAccess
- CloudWatchLogsFullAccess
- AmazonSSMReadOnlyAccess
- AWSCodeBuildAdminAccess

Key Concepts Learned

CodePipeline: Orchestrates the CI/CD stages. Connects source, build, and deploy. Stores artifacts in S3 between stages. Does not do the actual building or deploying itself.

CodeBuild: Executes commands in a managed container. Reads buildspec.yml for instructions. Used for both build AND deploy stages in this project.

buildspec.yml: YAML file that tells CodeBuild what to do. Has phases: install, pre_build, build, post_build. Artifacts section defines what to pass to next stage.

SSM Parameter Store: AWS service for storing configuration and secrets. SecureString type encrypts the value. Preserves exact formatting unlike environment variables. Accessed via AWS CLI in CodeBuild.

systemd: Linux service manager. Manages long-running processes. Keeps app running after SSH disconnects. Auto-restarts on crash. Commands: systemctl start/stop/restart/status/enable.

CodeConnections: AWS service that manages connections to GitHub. Required for CodePipeline to pull source from GitHub. Must have correct IAM permissions to use.

Final Verification

After full pipeline run:

```
curl http://localhost:5000/health
```

Output:

```
{
  "host": "ip-10-0-8-109.ec2.internal",
  "status": "healthy",
  "timestamp": "2026-06-06T11:52:37.970446",
  "uptime": "0h 0m 7s",
  "version": "1.0.1"
}
```

Version 1.0.1 confirmed — deployed automatically through the full pipeline.

What To Do Next

- Fix the auto-trigger issue by switching to GitHub Actions for source trigger
- Add pytest back to the build stage properly
- Add HTTPS via ALB + ACM certificate
- Move to a production WSGI server (Gunicorn) instead of Flask dev server
- Expand the API with more endpoints for the full finance project